8
Backtracking

Modern computers are so fast that brute force can be an effective and honorable way to solve problems. For example, sometimes it is easier to count the number of items in a set by actually constructing them than by using sophisticated combinatorial arguments. Of course, this requires the number of items searched to be small enough for the computation to complete.

A modern personal computer has a clock rate of about 1 gigahertz, meaning one billion operations per second. Figure that doing anything interesting takes a few hundred instructions or even more. Thus you can hope to search a few million items per second on contemporary machines.

It is important to realize how big (or how small) one million is. One million permutations means all arrangements of roughly 10 or 11 objects, but not more. One million subsets means all combinations of roughly 20 items, but not more. Solving significantly larger problems requires carefully pruning the search space to make sure we look at only the elements which really matter.

In this chapter, we look at backtracking algorithms for exhaustive search and designing effective pruning techniques to make them as powerful as possible.

8.1 Backtracking

Backtracking is a systematic method to iterate through all the possible configurations of a search space. It is a general algorithm/technique which must be customized for each individual application.

In the general case, we will model our solution as a vector $a = (a_1, a_2, ..., a_n)$, where each element a_i is selected from a finite ordered set S_i. Such a vector might represent an arrangement where a_i contains the ith element of the permutation. Or the vector might represent a given subset S, where a_i is true if and only if the ith element of the universe is in S. The vector can even represent a sequence of moves in a game or a path in a graph, where a_i contains the ith event in the sequence.

At each step in the backtracking algorithm, we start from a given partial solution, say, $a = (a_1, a_2, ..., a_k)$, and try to extend it by adding another element at the end. After extending it, we must test whether what we have so far is a solution – if so, we should print it, count it, or do what we want with it. If not, we must then check whether the partial solution is still potentially extendible to some complete solution. If so, recur and continue. If not, we delete the last element from a and try another possibility for that position, if one exists.

The honest working code is given below. We include a global `finished` flag to allow for premature termination, which could be set in any application-specific routine.

```
bool finished = FALSE;                /* found all solutions yet? */

backtrack(int a[], int k, data input)
{
        int c[MAXCANDIDATES];         /* candidates for next position */
        int ncandidates;              /* next position candidate count */
        int i;                        /* counter */

        if (is_a_solution(a,k,input))
                process_solution(a,k,input);
        else {
                k = k+1;
                construct_candidates(a,k,input,c,&ncandidates);
                for (i=0; i<ncandidates; i++) {
                        a[k] = c[i];
                        backtrack(a,k,input);
                        if (finished) return;   /* terminate early */
                }
        }
}
```

The application-specific parts of this algorithm consists of three subroutines:

- `is_a_solution(a,k,input)` — This Boolean function tests whether the first k elements of vector a are a complete solution for the given problem. The last argument, `input`, allows us to pass general information into the routine. We will use it to specify n, the size of a target solution. This makes sense when constructing permutations of size n or subsets of n elements, but may not be relevant when constructing variable-sized objects such as sequences of moves in a game. In such applications, this last argument can be ignored.

- `construct_candidates(a,k,input,c,ncandidates)` — This routine fills an array c with the complete set of possible candidates for the kth position of a, given the contents of the first $k-1$ positions. The number of candidates returned in this array is denoted by `ncandidates`. Again, `input` may be used to pass auxiliary information, particularly the desired solution-size.
- `process_solution(a,k)` — This routine prints, counts, or somehow processes a complete solution once it is constructed. Note that the auxiliary `input` is unnecessary here, since k denotes the number of elements in the solution.

Backtracking ensures correctness by enumerating all possibilities. It ensures efficiency by never visiting a state more than once.

Study how recursion yields an elegant and easy implementation of the backtracking algorithm. Because a new candidates array c is allocated with each recursive procedure call, the subsets of not-yet-considered extension candidates at each position will not interfere with each other. We will see that depth-first traversal in graphs (Chapter 9) uses essentially the same recursive algorithm as `backtrack`. Backtracking can be thought of as depth-first search over an implicit graph instead of an explicit one.

We now provide two examples of backtracking in action, by giving specific implementations of these three functions which iterate through all subsets and permutations of n elements.

8.2 Constructing All Subsets

As mentioned above, we can construct the 2^n subsets of n items by iterating through all possible 2^n length-n vectors of *true* or *false*, letting the ith element denote whether item i is or is not in the subset.

Using the notation of the general backtrack algorithm, $S_k = (true, false)$, and a is a solution whenever $k \geq n$. We can now construct all subsets with simple implementations for `is_a_solution()`, `construct_candidates()`, and `process_solution()`. Printing each subset out after constructing it actually proves to be the most complicated of the three routines!

```
is_a_solution(int a[], int k, int n)
{
        return (k == n);                /* is k == n? */
}

construct_candidates(int a[], int k, int n, int c[], int *ncandidates)
{
        c[0] = TRUE;
        c[1] = FALSE;
        *ncandidates = 2;
}
```

```
process_solution(int a[], int k)
{
        int i;                          /* counter */

        printf("{");
        for (i=1; i<=k; i++)
                if (a[i] == TRUE) printf(" %d",i);

        printf(" }\n");
}
```

Finally, we must instantiate the call to `backtrack` with the right arguments. Specifically, this means giving a pointer to the empty solution vector, setting $k = 0$ to denote that it is empty, and specifying the number of elements in the universal set:

```
generate_subsets(int n)
{
        int a[NMAX];                    /* solution vector */

        backtrack(a,0,n);
}
```

In what order will this routine generate the subsets of $\{1, 2, 3\}$? The critical issue is the order of moves generated in `construct_candidates`. Because *true* always appears before *false*, the subset of all trues is generated first, and the all-false empty set is generated last:

```
{ 1 2 3 }
{ 1 2 }
{ 1 3 }
{ 1 }
{ 2 3 }
{ 2 }
{ 3 }
{ }
```

8.3 Constructing All Permutations

The problem of constructing all permutations is similar to that of generating all subsets, except that the candidates for the next move now depend on the values in the partial solution. To avoid repeating permutation elements, we must ensure that the ith element of the permutation is distinct from all the elements before it.

To use the notation of the general backtrack algorithm, $S_k = \{1, \ldots, n\} - a$, and a is a solution whenever $k = n$:

```
construct_candidates(int a[], int k, int n, int c[], int *ncandidates)
{
        int i;                          /* counter */
        bool in_perm[NMAX];             /* who is in the permutation? */

        for (i=1; i<NMAX; i++) in_perm[i] = FALSE;
        for (i=0; i<k; i++) in_perm[ a[i] ] = TRUE;

        *ncandidates = 0;
        for (i=1; i<=n; i++)
                if (in_perm[i] == FALSE) {
                        c[ *ncandidates] = i;
                        *ncandidates = *ncandidates + 1;
                }
}
```

Testing whether i is a candidate for the kth slot in the permutation can be done by iterating through all $k-1$ elements of a and verifying that none of them matched, but we prefer to set up a bit-vector data structure (see Chapter 2) to maintain which elements are in the partial solution. This gives a constant-time legality check.

Completing the job of generating permutations requires specifying `process_solution` and `is_a_solution`, as well as setting the appropriate arguments to `backtrack`. All are essentially the same as for subsets:

```
process_solution(int a[], int k)
{
        int i;                          /* counter */

        for (i=1; i<=k; i++) printf(" %d",a[i]);

        printf("\n");
}

is_a_solution(int a[], int k, int n)
{
        return (k == n);
}

generate_permutations(int n)
{
        int a[NMAX];                    /* solution vector */

        backtrack(a,0,n);
}
```